

Моделирование клеточного автомата

1. Описание задания

Реализовать на языке Python один из одномерных клеточных автоматов (Правило 30, Правило 90, Правило 110 и др.). Построить визуализацию нескольких итераций эволюции автомата при различных начальных условиях.

2. Описание реализации

Для реализации клеточного автомата использован одномерный конечный автомат с дискретным временем, в котором каждое состояние клеток определяется по локальному правилу, зависящему от тройки соседних клеток. Были реализованы правила:

- Правило 30
- Правило 90
- Правило 110
- Правило 182

Каждое правило задается как отображение всех возможных троек (состояние левой, центральной и правой клетки) в новое состояние центральной клетки. Для преобразования номера правила (в диапазоне от 0 до 255) в булеву таблицу переходов использован бинарный код длиной 8 бит.

```
import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline

# Функции для вычислений

def rule_to_dict(rule_number: int) -> dict:
    """
    Преобразует номер правила (0-255) в отображение из 3-битовой окрестности
    в новое состояние клетки (0 или 1).
    """
    # Переводим число в 8-битное двоичное представление
    bin_str = f"{rule_number:08b}"
    neighborhoods = [
        (1,1,1), (1,1,0), (1,0,1), (1,0,0),
        (0,1,1), (0,1,0), (0,0,1), (0,0,0),
    ]
    # Сопоставляем каждой тройке соответствующее значение из бита правила
    return {nb: int(bit) for nb, bit in zip(neighborhoods, bin_str)}

# Функция моделирования клеточного автомата
def run_1d_ca(initial: np.ndarray, rule_dict: dict, steps: int) -> np.ndarray:
    """
    Запускает одномерный клеточный автомат:
    initial — начальный вектор (массив нулей и единиц),
    rule_dict — отображение окрестности -> новое состояние,
    steps — число шагов эволюции.
    """
```

```

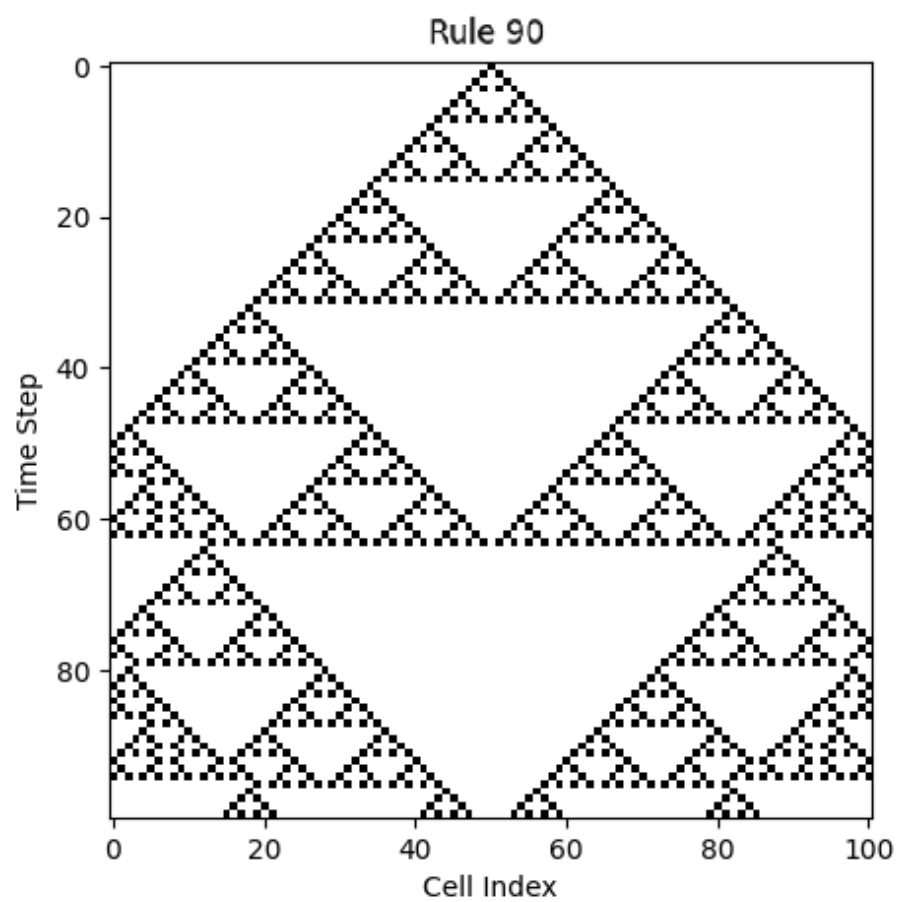
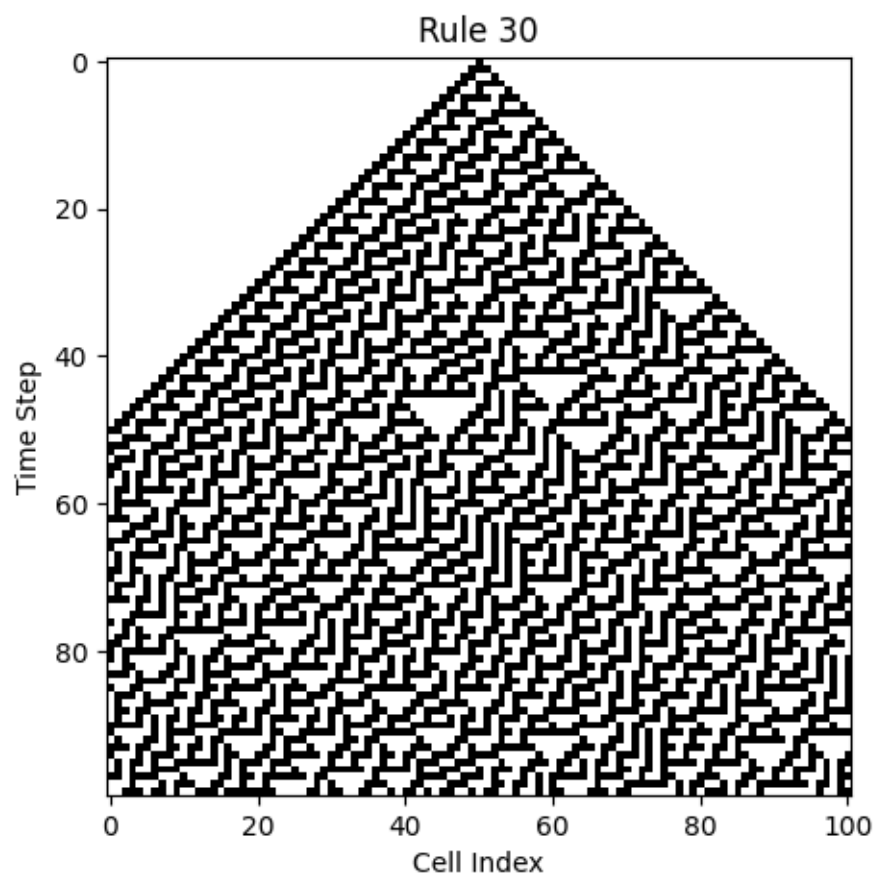
Возвращает матрицу history размером (steps × width).
"""
width = initial.size
history = np.zeros((steps, width), dtype=int)
history[0] = initial
for t in range(1, steps):
    prev = history[t-1]                # предыдущее состояние
    new = np.zeros_like(prev)          # новое состояние
    for i in range(width):
        nb = (prev[(i-1)%width], prev[i], prev[(i+1)%width])
        new[i] = rule_dict[nb]         # применяем правило
    history[t] = new                   # сохраняем результат
return history

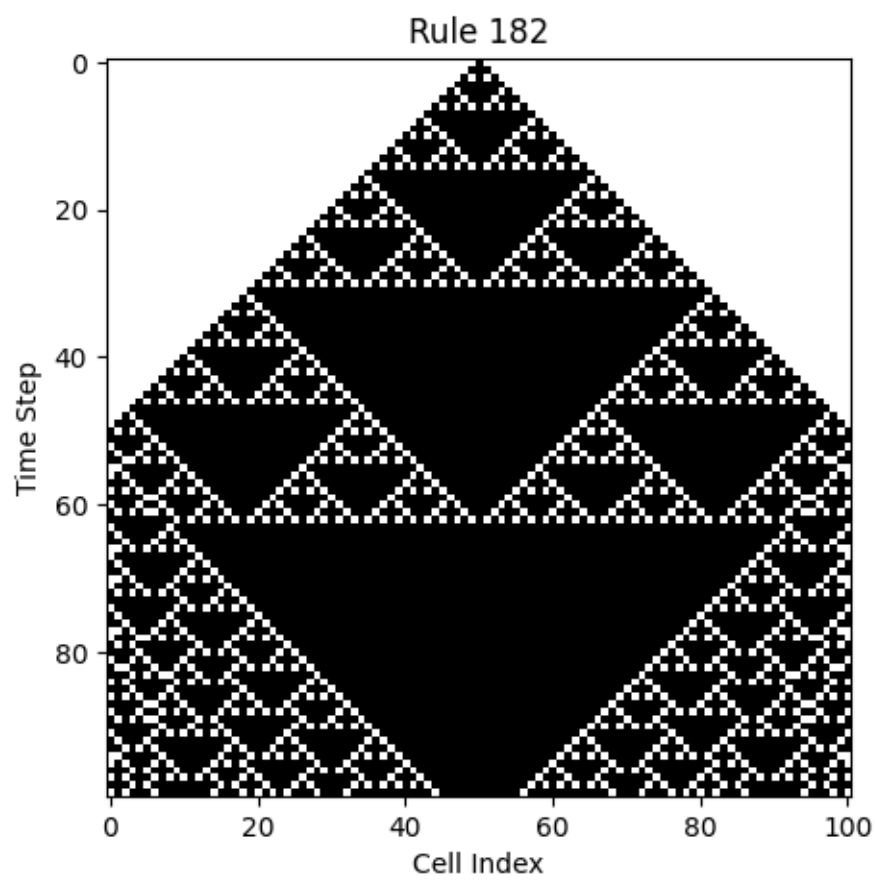
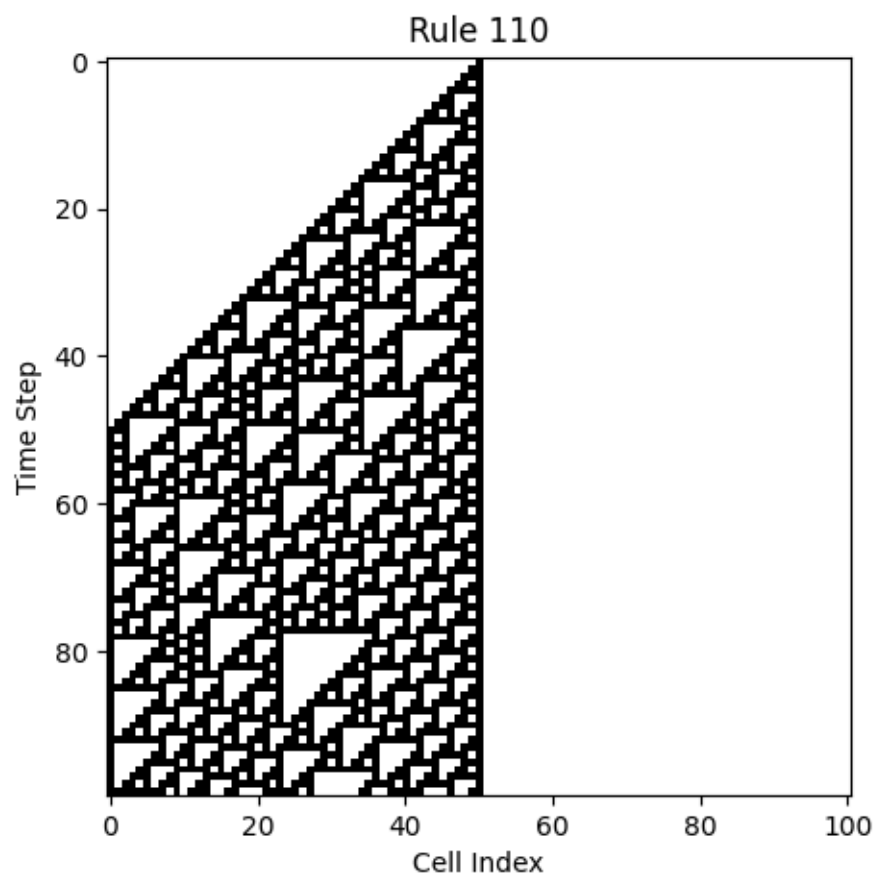
# Функция визуализации результата
def plot_ca(history: np.ndarray, title: str = None):
    """
    Строит эволюцию: по вертикали — время, по горизонтали — индекс клетки.
    """
    plt.figure(figsize=(5, 5))
    plt.imshow(history, cmap='binary', interpolation='nearest')
    plt.xlabel('Cell Index')
    plt.ylabel('Time Step')
    if title:
        plt.title(title)
    plt.show()

# Параметры симуляции
width = 101          # число клеток в ряду
steps = 100          # число шагов эволюции
rules = [30, 90, 110, 182] # список правил, которые хотим прогнать

# Основной цикл по правилам
for rule_number in rules:
    rule_map = rule_to_dict(rule_number)
    history = run_1d_ca(initial, rule_map, steps, boundary=bmode)
    plot_ca(history, title=f'Rule {rule_number}')

```





3. Вывод

В результате выполнения лабораторной работы была реализована модель одномерного клеточного автомата с возможностью выбора любого из 256 правил. Были построены графические визуализации, демонстрирующие эволюцию клеток во времени при правилах.